

# Software Requirements Specification

YourPortal — Job Portal Platform

---

Version 1.0 — Draft for Implementation

02 May 2026

Reference architecture: Naukri.com (verified May 2026)

# Table of Contents

## Software Requirements Specification

### YourPortal — Job Portal Platform

### Document Control

## 1. Introduction

### 1.1 Purpose

### 1.2 Document Scope

### 1.3 Intended Audience

### 1.4 Project Scope

In scope

Out of scope (initial release)

Deferred to Phase 2

### 1.5 Definitions, Acronyms, Abbreviations

### 1.6 References

## 2. Overall Description

### 2.1 Product Perspective

### 2.2 Product Functions Summary

### 2.3 User Classes and Characteristics

### 2.4 Operating Environment

Client side

Server side

### 2.5 Design and Implementation Constraints

### 2.6 User Documentation

### 2.7 Assumptions and Dependencies

## 3. External Interface Requirements

### 3.1 User Interfaces

Visual design language

Page-type catalog

Accessibility

### 3.2 Hardware Interfaces

### 3.3 Software Interfaces

### 3.4 Communications Interfaces

## 4. System Features (Functional Requirements)

### 4.1 Job Search & Discovery

Description

Functional requirements

Acceptance criteria

### 4.2 Job Detail & Application

Description

Functional requirements

Acceptance criteria

### 4.3 User Profile & Resume

Description

Functional requirements

Acceptance criteria

#### **4.4 Saved Jobs**

#### **4.5 Job Alerts**

Description

Functional requirements

Acceptance criteria

#### **4.6 Application Tracking**

Description

Functional requirements

#### **4.7 Company Directory & Profile**

Description

Functional requirements

#### **4.8 Career Advice (CMS)**

Description

Functional requirements

#### **4.9 Recruiter — Job Posting**

Description

Functional requirements

#### **4.10 Recruiter — Resume Database (Resdex equivalent)**

Description

Functional requirements

#### **4.11 Premium Services & Subscriptions ★**

Description

Functional requirements

Acceptance criteria

#### **4.12 Authentication & Authorization**

Description

Functional requirements

Acceptance criteria

#### **4.13 Notifications**

Description

Functional requirements

#### **4.14 Search Infrastructure**

Description

Functional requirements

#### **4.15 SEO & Sitemap Generation**

Functional requirements

#### **4.16 Admin Console & Feature Flags**

Description

Functional requirements

### **5. Non-functional Requirements**

- 5.1 Performance
- 5.2 Security
- 5.3 Reliability & Availability
- 5.4 Maintainability
- 5.5 Portability
- 5.6 Scalability
- 5.7 Usability & Accessibility
- 5.8 Internationalization
- 6. URL Architecture & Rendering Strategy
  - 6.1 URL Slug Taxonomy
    - Naming rules
    - Pattern catalog
  - 6.2 SSR/SSG/ISR/CSR Decision Matrix
  - 6.3 Canonicalization Rules
  - 6.4 Sitemap Strategy
- 7. Subscription & Feature Flag System
  - 7.1 Goals
  - 7.2 Conceptual Model
  - 7.3 Flag Types
  - 7.4 Data Model
  - 7.5 Evaluation Flow
  - 7.6 Caching Strategy
  - 7.7 Admin UI
  - 7.8 Initial Freemium State (Launch Day Configuration)
  - 7.9 Lifecycle: Enabling a Paid Service
    - Step 0: Prerequisites
    - Step 1: Internal QA test (USERTARGETED)
    - Step 2: Beta cohort (COHORTTARGETED)
    - Step 3: 10% rollout
    - Step 4: 50%, 100%
    - Step 5: General availability
  - 7.10 Subscription Lifecycle
  - 7.11 Payment Integration
  - 7.12 Entitlement Enforcement
    - Layer 1: Middleware (route gate)
    - Layer 2: Page (server component)
    - Layer 3: API (server-only — non-bypassable)
  - 7.13 Audit & Observability
  - 7.14 Worked Example: Launch Day → 6 Months Later
- 8. Data Model
  - 8.1 Entity Relationship Overview
  - 8.2 Schema Reference
    - Core entities

Feature flag entities

## 9. System Architecture

9.1 Layered Architecture

9.2 Deployment Topology

9.3 Tech Stack

9.4 Data Flow — Sample Request

## 10. Project Structure

10.1 Monorepo Layout

10.2 App Boundaries

10.3 Package Dependencies

## 11. Deployment & Operations

11.1 Hosting

11.2 CI/CD Pipeline

`.github/workflows/ci.yml`

11.3 Monitoring & Observability

11.4 Disaster Recovery

## 12. Roadmap

12.1 Phase 1 — Freemium MVP (Months 0-3)

12.2 Phase 2 — Subscription Launch (Months 3-6)

12.3 Phase 3 — Enterprise & Scale (Months 6-12)

## Appendix A — Glossary

## Appendix B — Naukri.com Reference Findings (Verified May 2026)

## Appendix C — Risk Register

## Appendix D — Compliance Checklist

## Appendix E — Sample Feature Flag Configurations

E.1 Launch day — minimal flag set (auto-seeded by pnpm db:seed)

E.2 Internal QA for Resume Display

E.3 10% percentage rollout of new homepage variant

E.4 Premium-tier-only feature

E.5 Emergency killswitch

E.6 Subscription launch sequence (when ready)

Order matters: turn on infra flags first, then make plans visible.

1. Master switch

2. Make pricing page accessible

3. Enable specific plan

4. Mark plan as sellable in DB

5. Enable features included in the plan (TIERGATED)

... etc

6. Verify

# Software Requirements Specification

## YourPortal — Job Portal Platform

|                              |                                                         |
|------------------------------|---------------------------------------------------------|
|                              |                                                         |
| Document version             | 1.0                                                     |
| Status                       | Draft for Implementation                                |
| Date                         | 02 May 2026                                             |
| Author                       | Project Owner                                           |
| Reference architecture       | Naukri.com (verified May 2026)                          |
| Prepared in conformance with | IEEE Std 830-1998 (adapted for modern web architecture) |

## Document Control

| Version | Date        | Author        | Changes                                                                                                                                                                  |
|---------|-------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 02-May-2026 | Project Owner | Initial baseline. Includes complete URL taxonomy, SSR/CSR strategy, backend-controlled subscription system, project folder structure, and Naukri.com reference findings. |

## Table of Contents

1. Introduction

2. Overall Description

3. External Interface Requirements

4. System Features (Functional Requirements)

5. Non-functional Requirements

6. URL Architecture & Rendering Strategy

7. Subscription & Feature Flag System

8. Data Model

9. System Architecture

10. Project Structure

11. Deployment & Operations

12. Roadmap

Appendix A — Glossary

Appendix B — Naukri.com Reference Findings

Appendix C — Risk Register

Appendix D — Compliance Checklist

Appendix E — Sample Feature Flag Configurations

# 1. Introduction

---

## 1.1 Purpose

---

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **YourPortal**, an India-focused job-search and recruitment platform. The document specifies what the system must do, the constraints under which it must operate, and how it is to be architected, deployed, and evolved.

The SRS serves three audiences:

1. **Engineering** — as the source of truth for what to build and how it fits together.
2. **Business / product** — as the contract for scope and prioritization.
3. **Future maintainers** — as a reference for why decisions were made.

## 1.2 Document Scope

---

This document covers:

- The complete public job-seeker site (search, browse, apply, profile, alerts).
- The recruiter portal (job posting, applicant tracking, optional resume search).
- Premium services offerings and subscription billing.
- The admin console and the **backend-controlled feature flag system** that gates all paid functionality.
- URL architecture, rendering strategy (SSR/SSG/ISR/CSR), and SEO requirements.
- The data model, system architecture, deployment topology, and operational concerns.

The document is implementation-aware: a working Next.js 15 monorepo scaffold accompanies this SRS at `Research/my-job-portal/`. Where this SRS describes a behaviour, the corresponding code path is referenced.

## 1.3 Intended Audience

---

| Audience                     | What they will use this document for                |
|------------------------------|-----------------------------------------------------|
| Backend / frontend engineers | Functional specs, data model, API contracts         |
| DevOps / SRE                 | Hosting topology, deployment, observability         |
| Product manager              | Feature inventory, roadmap, success metrics         |
| QA / test engineer           | Acceptance criteria, edge cases                     |
| Designer                     | User flows, page-type catalog, component priorities |
| Compliance / legal           | Privacy, data retention, audit trails               |
| External auditor             | Architecture overview, security baseline            |

## 1.4 Project Scope

---

### In scope

- Web (desktop + mobile-responsive) job-seeker product.
- Web recruiter portal.
- Backend API (BFF) shared across all clients.
- Admin console (feature flags, plans, users, jobs, companies, articles).
- Subscription billing with feature-flag-driven activation.
- SEO infrastructure (sitemap, JSON-LD, canonical URLs).
- Email notifications (transactional + alert digests).

### Out of scope (initial release)

- Native mobile apps (iOS / Android).
- Public REST/GraphQL API for third-party integrations.
- ATS integrations with employer HRIS systems.
- Video interviewing or live chat.
- Internationalization beyond Indian English.
- WhatsApp / SMS notifications.
- Resume parsing AI (extracted as separate module for Phase 2).

### Deferred to Phase 2

- Subscription system **activation** (system is built but every flag is OFF on launch — see §7).
- Premium services activation (Resume Display, Resume Writing, AI Interview, etc.).
- Recruiter resume database (Resdex equivalent).

## 1.5 Definitions, Acronyms, Abbreviations

---

| Term           | Meaning                                                                     |
|----------------|-----------------------------------------------------------------------------|
| <b>BFF</b>     | Backend-for-Frontend. Single API service shared by all client surfaces.     |
| <b>CSR</b>     | Client-Side Rendering. HTML is empty; JS renders content in the browser.    |
| <b>CSP</b>     | Content Security Policy. HTTP header restricting resource origins.          |
| <b>ISR</b>     | Incremental Static Regeneration. SSG with periodic revalidation.            |
| <b>JD</b>      | Job Description (also: a Job Detail page).                                  |
| <b>JSON-LD</b> | JSON for Linked Data. Schema.org structured data for SEO.                   |
| <b>MAU</b>     | Monthly Active User.                                                        |
| <b>PPR</b>     | Partial Pre-rendering (Next.js 15 feature: static shell + dynamic islands). |



| Term        | Meaning                                                                                 |
|-------------|-----------------------------------------------------------------------------------------|
| <b>RSC</b>  | React Server Component. Renders on server, no JS shipped to browser for that component. |
| <b>RUM</b>  | Real User Monitoring.                                                                   |
| <b>SSG</b>  | Static Site Generation. HTML built at build time.                                       |
| <b>SSR</b>  | Server-Side Rendering. HTML built at request time.                                      |
| <b>SRP</b>  | Search Results Page.                                                                    |
| <b>WCAG</b> | Web Content Accessibility Guidelines.                                                   |

## 1.6 References

---

1. Naukri.com Architecture Reverse-Engineering Report — [Research/naukri-deep-dive-analysis.md](#)
  2. Naukri Engineering blog: [medium.com/naukri-engineering](#)
  3. IEEE Std 830-1998 — Software Requirements Specifications
  4. Schema.org JobPosting specification — [schema.org/JobPosting](#)
  5. Google for Jobs developer guidelines — [developers.google.com/search/docs/appearance/structured-data/job-posting](#)
  6. Next.js 15 documentation — [nextjs.org/docs](#)
  7. WCAG 2.2 — [www.w3.org/TR/WCAG22/](#)
-

## 2. Overall Description

### 2.1 Product Perspective

YourPortal is a greenfield product directly modeled on the patterns established by Naukri.com (India's #1 job site, 188M page views/day). The core value proposition is a search-and-discovery surface for jobs across all Indian cities and skills, paired with employer features for job posting and (post-launch) candidate sourcing.

The product is **API-centric**: a single NestJS BFF serves all front-end clients (job-seeker site, recruiter portal, services site, future mobile apps). All paid functionality is gated by a backend feature flag system so that **monetization can be turned on per-feature when the business is ready, without frontend deploys**.

### 2.2 Product Functions Summary

The system provides:

1. **Public job search** — by skill, city, designation, industry, salary, experience.
2. **Job detail pages** with structured data for Google for Jobs.
3. **User profiles** with resume upload and parsing.
4. **Job applications** with status tracking from APPLIED → HIRED.
5. **Saved jobs** and **job alerts** (daily/weekly email digests).
6. **Company directory** with profile pages, "why join us", reviews.
7. **Career advice** (CMS-managed editorial content).
8. **Recruiter portal** for job posting and applicant tracking.
9. **Premium services** (initially OFF; admin enables them when ready).
10. **Subscription billing** (initially OFF; admin enables when ready).
11. **Admin console** with feature-flag toggles, plan management, user lookup.

### 2.3 User Classes and Characteristics

| Class                    | Description                                                 | Auth                                          | Primary device                  |
|--------------------------|-------------------------------------------------------------|-----------------------------------------------|---------------------------------|
| <b>Anonymous Visitor</b> | Searches, reads articles, reads company profiles            | None                                          | Mobile web (60%), desktop (40%) |
| <b>Candidate</b>         | Applies to jobs, manages profile, gets alerts               | Email + password OR OAuth                     | Mobile web (70%), desktop (30%) |
| <b>Premium Candidate</b> | Same as Candidate + paid features                           | Same + active subscription                    | Same                            |
| <b>Recruiter</b>         | Posts jobs, reviews applicants, optionally searches resumes | Email + password (separate registration flow) | Desktop (95%)                   |
| <b>Admin</b>             | Manages feature flags, plans, users, content                | Email + password + role=ADMIN                 | Desktop                         |

Estimated mix at year 1: 65% anonymous, 30% candidate (free), 1-3% premium candidate, 2-5% recruiter.

## 2.4 Operating Environment

### Client side

- Modern browsers: Chrome 120+, Edge 120+, Safari 17+, Firefox 120+.
- Mobile: iOS Safari, Android Chrome on devices  $\geq$  2GB RAM (covering ~95% of Indian smartphone users).
- Slow network targets: usable on 3G; LCP  $<$  4s on Slow 3G; bundle  $\leq$  150KB JS first-load.

### Server side

- Vercel for Next.js apps (auto-scaled serverless functions).
- Render or Fly.io for NestJS BFF (long-running container).
- Neon (managed Postgres) — Singapore region.
- Cloudflare for CDN + DNS + WAF.
- All TLS 1.3, HSTS-preloaded, max-age=2yr.

## 2.5 Design and Implementation Constraints

| #   | Constraint                                                     | Reason                                               |
|-----|----------------------------------------------------------------|------------------------------------------------------|
| C1  | TypeScript end-to-end (frontends + BFF + tooling)              | One language, shared types, no FFI overhead          |
| C2  | All paid features OFF on launch                                | User has confirmed: starting freemium                |
| C3  | No frontend deploy required to enable a feature                | Backend-controlled flag system is mandatory          |
| C4  | All public pages must serve indexable HTML                     | SEO is the primary growth channel                    |
| C5  | All job detail pages must include Schema.org JobPosting        | Required for Google for Jobs                         |
| C6  | Numeric ID at end of every detail-page URL                     | Robust to slug drift; matches Naukri pattern         |
| C7  | Full-text search via Meilisearch (not Elasticsearch initially) | Operational simplicity at our scale                  |
| C8  | Standard JWT + httpOnly refresh cookie                         | Industry standard; not Naukri's custom header scheme |
| C9  | All UI must meet WCAG 2.1 Level AA                             | Inclusion + legal                                    |
| C10 | All pages must respond within performance budgets (\$5.1)      | Quality bar                                          |

## 2.6 User Documentation

The user-facing help content lives in the CMS ( `/career-advice/...` ) and is editorial, not technical. The internal documentation tree:

- `Research/naukri-deep-dive-analysis.md` — reverse-engineering report
- `Research/SRS.md` — this document

- [Research/my-job-portal/README.md](#) — repo overview
- [Research/my-job-portal/docs/architecture.md](#) — system design
- [Research/my-job-portal/docs/ssr-csr-decisions.md](#) — rendering choices
- [Research/my-job-portal/docs/url-taxonomy.md](#) — URL patterns
- [Research/my-job-portal/docs/subscription-system.md](#) — feature-flag deep-dive
- [Research/my-job-portal/docs/database-schema.md](#) — Prisma schema explained
- [Research/my-job-portal/docs/deployment.md](#) — hosting topology

## 2.7 Assumptions and Dependencies

---

### Assumptions:

1. The launch market is India; UI is English-first; ₹INR is the only billing currency.
2. The team has Node.js + TypeScript skills; no Java, PHP, or Python knowledge required for app code.
3. Cloudflare and Vercel free tiers are sufficient until 100k MAU.
4. Stripe (or Razorpay) will be the payment provider for subscriptions (when activated).
5. Email sending will use Resend (or similar transactional provider) — not self-hosted SMTP.

### Dependencies:

1. Domain registration ( [your-domain.com](#) ) and DNS configuration.
  2. SSL certificates (auto-issued by Cloudflare).
  3. Postgres managed instance (Neon).
  4. Object storage for resumes and logos (Cloudflare R2).
  5. Email provider account (Resend).
  6. Sentry account for error tracking.
  7. PostHog account for product analytics.
  8. Stripe / Razorpay merchant account (only required when subscriptions activate).
-

## 3. External Interface Requirements

### 3.1 User Interfaces

#### Visual design language

Modeled on Naukri's design system findings (Bootcamp/Medium article):

- **Brand archetype:** The Sage (TED, Google, BBC reference set)
- **Palette:** Soft blue `rgb(10, 102, 194)` primary; teal `rgb(20, 184, 166)` secondary
- **Typography:** Inter (sans-serif primary); optional serif accent for editorial
- **Spacing:** 4px grid; container max-width 1200px
- **Components prioritized:** Inputs, selection controls, CTA buttons, cards, "belly filters"
- **Built in:** Figma; documented as React component library at `packages/ui`

#### Page-type catalog

| Surface             | Examples                                                                                             | Rendering                   | SEO-critical |
|---------------------|------------------------------------------------------------------------------------------------------|-----------------------------|--------------|
| Homepage            | <code>/</code>                                                                                       | PPR                         | ✓            |
| SEO landings        | <code>/python-jobs</code> , <code>/jobs-in-bangalore</code> , <code>/python-jobs-in-pune</code>      | SSR + edge cache            | ✓✓✓          |
| Multi-city landings | <code>/jobs-in-mumbai-and-pune-and-bangalore</code>                                                  | SSR                         | ✓            |
| Job detail          | <code>/job/{slug}-{id}</code>                                                                        | SSR + JSON-LD               | ✓✓✓          |
| Companies directory | <code>/companies</code>                                                                              | SSR                         | ✓✓           |
| Company profile     | <code>/ {slug}-overview-{id}</code>                                                                  | SSR + canonicalization      | ✓✓           |
| Why join us         | <code>/working-at-{slug}-{id}</code>                                                                 | SSR                         | ✓            |
| Career advice       | <code>/career-advice</code> + <code>/career-advice/{slug}</code>                                     | SSR (index) + SSG (article) | ✓✓           |
| Auth                | <code>/login</code> , <code>/register</code> , <code>/forgot-password</code>                         | CSR (Client Component)      | noindex      |
| Dashboard           | <code>/profile</code> , <code>/applications</code> , <code>/saved-jobs</code> , <code>/alerts</code> | CSR behind SSR auth check   | noindex      |
| Admin               | <code>/admin/*</code>                                                                                | CSR                         | noindex      |
| Pricing             | <code>/pricing</code>                                                                                | SSR (404 when flag OFF)     | conditional  |
| Services            | <code>/services/{slug}</code>                                                                        | SSR (404 when flag OFF)     | conditional  |

| Surface        | Examples                                              | Rendering | SEO-critical |
|----------------|-------------------------------------------------------|-----------|--------------|
| Static / legal | /about , /contact , /privacypolicy , /termsconditions | SSG       | low          |
| Sitemap        | /sitemap.xml , /robots.txt                            | Generated | n/a          |

## Accessibility

- WCAG 2.1 Level AA target.
- All interactive elements keyboard-reachable.
- Focus rings visible.
- Color contrast  $\geq 4.5:1$  for body text,  $\geq 3:1$  for large text and graphics.
- Form fields with `<label>` , `aria-describedby` for errors.
- Skip link to main content on every page.
- `prefers-reduced-motion` respected.

## 3.2 Hardware Interfaces

None directly. The product is web-based; client devices interact only via standard HTTP(S) over the browser network stack.

## 3.3 Software Interfaces

| Service                       | Protocol              | Purpose                                            | Required at launch    |
|-------------------------------|-----------------------|----------------------------------------------------|-----------------------|
| PostgreSQL 16                 | Postgres wire         | Primary OLTP datastore                             | Yes                   |
| Meilisearch                   | HTTPS REST            | Full-text search index for jobs/companies/articles | Yes                   |
| Redis                         | RESP                  | Sessions + BullMQ job queues                       | Yes                   |
| Cloudflare R2 (S3-compatible) | HTTPS S3 API          | Resume/logo/article-image storage                  | Yes                   |
| Cloudflare CDN                | HTTP edge             | Public asset delivery + cache                      | Yes                   |
| Resend                        | HTTPS REST            | Transactional email                                | Yes (alerts can wait) |
| Sentry                        | HTTPS REST            | Error + performance monitoring                     | Yes                   |
| PostHog                       | HTTPS REST            | Product analytics + RUM                            | Yes                   |
| Stripe (or Razorpay)          | HTTPS REST + webhooks | Subscription billing                               | No (Phase 2)          |
| AmbitionBox-equivalent        | HTTPS (TBD)           | Reviews enrichment on company pages                | No (Phase 3)          |

## 3.4 Communications Interfaces

---

- All client/server traffic over HTTPS only (HSTS preload).
  - WebSockets reserved for future real-time recruiter messaging (not in initial scope).
  - Email outbound only (no inbound mail processing).
  - Webhook inbound from Stripe/Razorpay (Phase 2) — IP allowlist + signature verification.
-

## 4. System Features (Functional Requirements)

---

This section enumerates the functional requirements grouped by user-visible feature. Each feature has:

- A description.
- Functional requirements (numbered FR-X.Y).
- Acceptance criteria.

### 4.1 Job Search & Discovery

---

#### Description

Anonymous and authenticated users can find jobs by keyword, skill, location, designation, industry, experience, and salary. Browse pages are SEO-indexable.

#### Functional requirements

- **FR-4.1.1** — System SHALL provide a search input on every page that accepts free-text keywords and routes to the SRP.
- **FR-4.1.2** — System SHALL support filtering by: skill (multi-select), city (multi-select including multi-city - and- URL pattern), industry, functional area, employment type, work mode (on-site/hybrid/remote), experience min/max in years, salary min, posted-within (24h / 7d / 30d).
- **FR-4.1.3** — System SHALL provide sortable results: relevance (default), most recent, salary high-to-low.
- **FR-4.1.4** — System SHALL paginate results 20 per page; query param `?page=N`.
- **FR-4.1.5** — System SHALL serve unique SEO-friendly URLs per filter combination — see §6.1 URL Slug Taxonomy.
- **FR-4.1.6** — System SHALL render JSON-LD `ItemList + BreadcrumbList` on every SEO landing.
- **FR-4.1.7** — System SHALL support type-ahead suggestions from skills and cities (debounced 200ms).
- **FR-4.1.8** — System SHALL render canonical link header on every SRP.
- **FR-4.1.9** — Empty results SHALL show a helpful message with related searches.

#### Acceptance criteria

- A non-existent slug returns 404 (not 200 with empty page).
- The same set of filters always produces the same canonical URL (alphabetical city order; query params sorted).
- A multi-city URL `/jobs-in-mumbai-and-pune` returns a 200 page combining both city result sets.
- The search input on the homepage prefetches the SRP route on focus.



## 4.2 Job Detail & Application

---

### Description

Each job has a permanent detail page with full description, employer info, apply CTA, similar jobs, and structured data for Google for Jobs.

### Functional requirements

- **FR-4.2.1** — System SHALL serve job detail at `/job/{seo-slug}-{numericId}` where `{numericId}` is the database primary key.
- **FR-4.2.2** — System SHALL inject Schema.org `JobPosting` JSON-LD with: title, description, datePosted, validThrough, employmentType, hiringOrganization (with logo), jobLocation, baseSalary (when not hidden), experienceRequirements, identifier, url, directApply.
- **FR-4.2.3** — When the URL `{seo-slug}` does not match the canonical slug for `{numericId}`, the system SHALL 301-redirect to the canonical URL.
- **FR-4.2.4** — System SHALL show "Apply" CTA. For anonymous users, click triggers login flow with `?next=...` redirect-back.
- **FR-4.2.5** — Authenticated users with a complete profile SHALL be able to apply in one click; the application is recorded in the `Application` table with `status=APPLIED`.
- **FR-4.2.6** — System SHALL prevent duplicate applications (UNIQUE constraint on `(userId, jobId)`).
- **FR-4.2.7** — Closed/expired jobs SHALL render with `noindex` meta and a notice.
- **FR-4.2.8** — System SHALL show 8-12 similar jobs computed from shared skills and industry.
- **FR-4.2.9** — System SHALL provide a "Save" toggle for authenticated users (writes to `SavedJob`).
- **FR-4.2.10** — System SHALL implement Cloudflare cache rule: edge TTL 60s, SWR 1h.
- **FR-4.2.11** — System SHALL purge the cached page on job edit/close (Cloudflare Purge API).

### Acceptance criteria

- Google's Rich Results Test validates the JobPosting JSON-LD without errors.
- Clicking Apply while logged out lands on `/login?next=/job/...` and after login returns to the job page.
- Re-applying to the same job returns a friendly "already applied" message, not a 500 error.

## 4.3 User Profile & Resume

---

### Description

Authenticated candidates manage their profile, education, experience, skills, and resume.

### Functional requirements

- **FR-4.3.1** — System SHALL store profile fields: full name, email, phone, headline, summary, total experience (months), current company, current title, current salary, expected salary, notice period, preferred cities, skills.
- **FR-4.3.2** — System SHALL store education history (institute, degree, field, start year, end year).

- **FR-4.3.3** — System SHALL store work experience (company, title, dates, description, current).
- **FR-4.3.4** — System SHALL accept resume uploads in PDF or DOCX, max 5 MB; stored in R2 with signed URL access.
- **FR-4.3.5** — System SHALL display a "profile completeness" score (0-100) based on filled fields.
- **FR-4.3.6** — System SHALL allow the user to download their resume (when `feature.resume_download_pdf` is enabled for their tier).
- **FR-4.3.7** — System SHALL log every profile edit for audit (timestamp + diff).
- **FR-4.3.8** — Profile pages are `noindex`.

## Acceptance criteria

- Resume upload validates MIME type and size before accepting.
- Profile completeness recalculates immediately on edit (server-side).

## 4.4 Saved Jobs

---

- **FR-4.4.1** — System SHALL provide one-click save/unsave on every job card and detail page (auth required).
- **FR-4.4.2** — System SHALL provide `/saved-jobs` listing user's saved jobs with date saved, sortable.
- **FR-4.4.3** — System SHALL purge a user's saved entry when the underlying job is deleted (cascade FK).
- **FR-4.4.4** — System SHALL expose `/api/me/saved-jobs` for programmatic management.

## 4.5 Job Alerts

---

### Description

Users subscribe to recurring searches; the system emails them new matches daily/weekly.

### Functional requirements

- **FR-4.5.1** — System SHALL allow users to create up to 10 named alerts each with: query, skills, cities, experience range, min salary, frequency (instant / daily / weekly).
- **FR-4.5.2** — System SHALL allow active/paused state per alert.
- **FR-4.5.3** — System SHALL run a BullMQ daily worker that for each active alert: searches for new matches, deduplicates against last-sent jobIds, and enqueues an email if there are new matches.
- **FR-4.5.4** — Alert emails SHALL include: matched jobs (max 20), one-click unsubscribe link, link to manage all alerts.
- **FR-4.5.5** — System SHALL respect `killswitch.job_alerts` flag (emergency stop).

### Acceptance criteria

- An alert that matched no new jobs since last run does NOT send an email.
- Unsubscribing from one alert does NOT affect other alerts.

## 4.6 Application Tracking

---

### Description

Candidates see all their applications and current status.

### Functional requirements

- **FR-4.6.1** — System SHALL list applications at `/applications` filtered by status (default: all).
- **FR-4.6.2** — Status SHALL transition through: APPLIED → VIEWED → SHORTLISTED → INTERVIEW\_SCHEDULED → INTERVIEW\_COMPLETED → OFFER\_EXTENDED → HIRED OR REJECTED OR WITHDRAWN.
- **FR-4.6.3** — Recruiters SHALL transition status from their portal; candidates can WITHDRAW.
- **FR-4.6.4** — System SHALL email candidate on each status change.
- **FR-4.6.5** — System SHALL track `viewedByEmployerAt` timestamp.

## 4.7 Company Directory & Profile

---

### Description

Browseable directory of employers; each company has a profile page, "why join us" page, and aggregate stats.

### Functional requirements

- **FR-4.7.1** — System SHALL serve company directory at `/companies` paginated 24 per page, filterable by industry ( `?category=...` ).
- **FR-4.7.2** — System SHALL serve company profile at `/slug-overview-numericId` with: hero, about, current openings, reviews, ratings.
- **FR-4.7.3** — System SHALL serve "why join us" at `/working-at-slug-numericId` .
- **FR-4.7.4** — System SHALL inject Schema.org `Organization` + `BreadcrumbList` .
- **FR-4.7.5** — Company pages SHALL canonicalize slug drift via 301.
- **FR-4.7.6** — System SHALL implement Cloudflare cache rule: edge TTL 1h, SWR 6h.

## 4.8 Career Advice (CMS)

---

### Description

Editorial content for SEO and user education. Articles, lists, FAQs.

### Functional requirements

- **FR-4.8.1** — System SHALL serve article index at `/career-advice` with tag filter ( `?tag=...` ).

- **FR-4.8.2** — System SHALL serve article detail at `/career-advice/{slug}` with full body, author, related articles.
- **FR-4.8.3** — System SHALL inject Schema.org `Article` + `BreadcrumbList` + `FAQPage` (when FAQs present).
- **FR-4.8.4** — Articles SHALL be SSG (built at deploy time) for max performance; new articles trigger ISR rebuild.
- **FR-4.8.5** — Authoring SHALL be done in admin console (`/admin/articles`) with markdown editor.
- **FR-4.8.6** — System SHALL support DRAFT, PUBLISHED, ARCHIVED states.

## 4.9 Recruiter — Job Posting

---

### Description

Companies/recruiters post jobs and manage applicants.

### Functional requirements

- **FR-4.9.1** — Recruiters SHALL register via `/recruit/register` (separate flow from candidates).
- **FR-4.9.2** — Email-domain verification SHALL link recruiter to a Company record.
- **FR-4.9.3** — Recruiters SHALL post a job via wizard: title, description, skills, locations, experience, salary, employment type, work mode.
- **FR-4.9.4** — System SHALL enforce free-tier daily/monthly post limits via `feature.recruiter_post_quota` flag.
- **FR-4.9.5** — Recruiters SHALL view all their posted jobs at `/recruiter/jobs`, with filter by status.
- **FR-4.9.6** — Recruiters SHALL view applicants per job at `/recruiter/jobs/{id}/applicants` with applicant resume download, status update, notes.
- **FR-4.9.7** — Recruiter portal pages SHALL be `noindex`.

## 4.10 Recruiter — Resume Database (Resdex equivalent)

---

### Description

Premium recruiter feature: search the full candidate database. Available only when `recruiter.resdex.enabled` flag is ON AND recruiter has the entitlement.

### Functional requirements

- **FR-4.10.1** — When the flag is OFF, `/recruiter/resdex` SHALL return 404. (Initial state on launch.)
- **FR-4.10.2** — When ON, recruiters with entitlement SHALL search candidates by skills, location, experience, current salary range, current company, education.
- **FR-4.10.3** — System SHALL meter searches per recruiter per day; daily limit by tier.
- **FR-4.10.4** — Search results SHALL show masked candidate identity unless the recruiter "purchases contact" (uses one credit).
- **FR-4.10.5** — All Resdex actions SHALL be audit-logged.

## 4.11 Premium Services & Subscriptions ★

**THIS SECTION IS THE KEY DELTA from a typical job portal SRS. The entire premium services and subscription system is built but DEACTIVATED on launch. The admin enables it later via feature flags.**

### Description

Catalog of paid services (Resume Display, Resume Writing, AI Interview, etc.) and subscription plans (Basic, Premium, Enterprise). All controlled by backend flags.

### Functional requirements

#### General

- **FR-4.11.1** — On initial launch, the Services menu SHALL NOT appear in the header (`services.menu.visible` flag = OFF).
- **FR-4.11.2** — On initial launch, all individual service flags SHALL be OFF; users SHALL NOT be offered any paid service.
- **FR-4.11.3** — On initial launch, the `/pricing` page SHALL return 404 (`subscription.pricing_page.visible` flag = OFF).
- **FR-4.11.4** — On initial launch, no `SubscriptionPlan` SHALL be active or public.
- **FR-4.11.5** — All paid features SHALL behave as if the user is on FREE tier (i.e., no feature gating in effect; fall-back to free behaviour).

#### When admin activates a service or plan

- **FR-4.11.6** — Activating `services.menu.visible` AND any individual `services.{name}.enabled` flag SHALL cause the Services menu to appear in the header on the next page load (no deploy needed).
- **FR-4.11.7** — Activating a `SubscriptionPlan.isActive=true`, `isPublic=true` AND `subscription.plans.{slug}.enabled` flag SHALL cause that plan to appear on `/pricing`.
- **FR-4.11.8** — Activating `subscription.pricing_page.visible` SHALL cause `/pricing` to render (no longer 404).
- **FR-4.11.9** — Activating `subscription.system.enabled` SHALL turn on the entire billing subsystem (webhooks, checkout, etc.).

#### Service detail pages

- **FR-4.11.10** — System SHALL serve `/services/{slug}` for each enabled service.
- **FR-4.11.11** — When the service flag is OFF, the route SHALL return 404 (or redirect to home, depending on configuration).
- **FR-4.11.12** — Service pages SHALL show: hero, benefit list, pricing, customer testimonials (CMS-managed), FAQ, purchase CTA.

### Purchase flow

- **FR-4.11.13** — System SHALL integrate with Stripe (or Razorpay) for one-time and recurring payments.
- **FR-4.11.14** — On successful payment webhook, system SHALL: create/update `Subscription` row, refresh `UserEntitlement` cache, send confirmation email.
- **FR-4.11.15** — System SHALL handle failed payments by setting subscription `status=PAST_DUE` and emailing the user.

### Tier-based feature gating

- **FR-4.11.16** — `feature.unlimited_applications`, `feature.profile_views_insights`, etc. SHALL be evaluated per-request via `@portal/feature-flags`.
- **FR-4.11.17** — When the flag returns `false` for a user, the system SHALL fall back to the free-tier behaviour (e.g., enforce daily application limit).
- **FR-4.11.18** — Server-side enforcement is mandatory at every protected API endpoint; client-side gating is for UX only.

### Audit and rollback

- **FR-4.11.19** — Every flag change SHALL be recorded in `FlagAuditLog` with: before-state, after-state, user, timestamp, optional reason.
- **FR-4.11.20** — Admin SHALL be able to revert any flag to its previous value with one click in the admin console.

### Acceptance criteria

- On launch day (all flags OFF): no Services menu in header; `/pricing` returns 404; all `/services/*` routes return 404; users experience the portal as 100% free with no paywalls.
- After enabling `services.menu.visible` AND `services.resume_display.enabled`: Services menu appears with one item ("Resume Display"). Clicking it shows the Resume Display landing page. No other change to the rest of the site.
- After disabling a previously-active service: menu item disappears; existing customers retain entitlements until subscription expiry.

## 4.12 Authentication & Authorization

---

### Description

Email + password primary; OAuth (Google, LinkedIn) optional. Role-based access (Candidate, Recruiter, Admin).

### Functional requirements

- **FR-4.12.1** — Registration SHALL accept email, password ( $\geq 8$  chars, must include digit + special), name, optional phone.
- **FR-4.12.2** — Passwords SHALL be hashed using Argon2id with sane parameters (memory  $\geq 64$ MB, iterations  $\geq 3$ , parallelism 1).

- **FR-4.12.3** — Login SHALL issue a 15-minute JWT access token (httpOnly cookie) and a 30-day refresh token (httpOnly cookie).
- **FR-4.12.4** — Refresh token SHALL be rotated on every use; old token SHALL be invalidated.
- **FR-4.12.5** — System SHALL implement password reset via emailed one-time token (15-min validity).
- **FR-4.12.6** — System SHALL support OAuth via Google and LinkedIn (Phase 2).
- **FR-4.12.7** — Failed login attempts SHALL be rate-limited (5/min per IP, 10/hour per email).
- **FR-4.12.8** — Email verification SHALL be required for: applying to jobs, posting jobs, purchasing services.
- **FR-4.12.9** — Sessions table SHALL track active sessions per user; user SHALL be able to revoke any session.
- **FR-4.12.10** — Admin role SHALL be assigned manually via DB; never via UI.

## Acceptance criteria

- Plaintext passwords are never logged anywhere.
- Cookies are Secure; HttpOnly; SameSite=Lax .
- A leaked refresh token can be invalidated within 5 minutes via session revoke.

## 4.13 Notifications

---

### Description

Transactional and digest emails. SMS/WhatsApp deferred.

### Functional requirements

- **FR-4.13.1** — Transactional emails: registration confirmation, password reset, application status change, job-posted confirmation, payment receipt.
- **FR-4.13.2** — Digest emails: daily/weekly job alerts.
- **FR-4.13.3** — Every email SHALL include unsubscribe link (per CAN-SPAM / GDPR best practice).
- **FR-4.13.4** — System SHALL maintain a per-user email-preferences table.
- **FR-4.13.5** — Email sending SHALL go through BullMQ queue with retry-on-failure.

## 4.14 Search Infrastructure

---

### Description

Meilisearch-backed search across jobs, companies, articles.

### Functional requirements

- **FR-4.14.1** — System SHALL maintain three Meilisearch indexes: jobs , companies , articles .
- **FR-4.14.2** — jobs index SHALL include searchable: title, description, shortDescription, companyName, skills.

- **FR-4.14.3** — `jobs` index SHALL include filterable: `skillSlugs`, `citySlugs`, `cityIds`, `industrySlug`, `status`, `minExperienceMonths`, `salaryMin`, `salaryMax`.
- **FR-4.14.4** — `jobs` index SHALL include sortable: `postedAt`, `salaryMax`.
- **FR-4.14.5** — System SHALL ship a daily full-reindex script ( `packages/search/scripts/reindex.ts` ).
- **FR-4.14.6** — System SHALL refresh individual documents on data change (Postgres LISTEN/NOTIFY → BullMQ → Meilisearch).
- **FR-4.14.7** — System SHALL provide type-ahead via Meilisearch's prefix search.

## 4.15 SEO & Sitemap Generation

---

### Functional requirements

- **FR-4.15.1** — System SHALL generate `/sitemap.xml` listing: `home`, all `/skill-jobs`, all `/jobs-in-city`, all `/slug-overview-id`, all `/career-advice/slug`, recent `/job/slug-id` (last 30 days only).
- **FR-4.15.2** — When the sitemap exceeds 50,000 URLs, the system SHALL split into category-shards using `Next.js generateSitemaps`.
- **FR-4.15.3** — System SHALL serve `/robots.txt` allowing all crawlers (configurable: AI-bot blocks via flag).
- **FR-4.15.4** — System SHALL serve canonical URL per page via `<link rel="canonical">`.
- **FR-4.15.5** — System SHALL serve hreflang only when multi-language is added (Phase 3).
- **FR-4.15.6** — Closed/expired jobs SHALL render `<meta name="robots" content="noindex">`.

## 4.16 Admin Console & Feature Flags

---

### Description

Internal tools for managing flags, plans, users, content, and moderation.

### Functional requirements

- **FR-4.16.1** — Admin console SHALL be served at `/admin/*`, role-gated to `User.role=ADMIN`.
- **FR-4.16.2** — `/admin/feature-flags` SHALL list all flags grouped by category, each with a toggle.
- **FR-4.16.3** — Toggling a flag SHALL: write to `FeatureFlag`, write to `FlagAuditLog`, trigger Cloudflare cache purge for affected URL patterns.
- **FR-4.16.4** — `/admin/subscriptions` SHALL list all `SubscriptionPlan`s with active/public toggles, view subscriber counts.
- **FR-4.16.5** — `/admin/users` SHALL provide user search, role assignment, manual entitlement grant (with optional expiry).
- **FR-4.16.6** — `/admin/jobs` SHALL list pending-moderation jobs with approve/reject.
- **FR-4.16.7** — `/admin/companies` SHALL list pending-verification companies.
- **FR-4.16.8** — `/admin/articles` SHALL provide CMS for career-advice (markdown editor, draft/publish).
- **FR-4.16.9** — `/admin/audit-log` SHALL show recent changes across all entities.



## 5. Non-functional Requirements

### 5.1 Performance

| Metric                          | Target (p75)          | Tool                   |
|---------------------------------|-----------------------|------------------------|
| LCP (Largest Contentful Paint)  | < 2.5s                | Lighthouse, Web Vitals |
| INP (Interaction to Next Paint) | < 200ms               | Web Vitals             |
| CLS (Cumulative Layout Shift)   | < 0.1                 | Web Vitals             |
| First-load JS                   | < 150 KB gzipped      | next-bundle-analyzer   |
| API endpoint p95                | < 300ms               | Sentry tracing         |
| SSR origin TTFB p75             | < 500ms               | Cloudflare Analytics   |
| Search query p95                | < 100ms (Meilisearch) | direct measurement     |

### 5.2 Security

- HSTS preload, max-age=63072000, includeSubDomains
- Strict CSP: default-src 'self'; asset domains explicitly whitelisted
- X-Frame-Options: SAMEORIGIN; X-Content-Type-Options: nosniff
- Referrer-Policy: strict-origin-when-cross-origin
- All cookies: HttpOnly; Secure; SameSite=Lax
- Argon2id for passwords (memory ≥64MB)
- JWT signed HS256; refresh tokens rotated on use
- Rate-limit (Cloudflare WAF + NestJS @Throttler): public 100/min/IP, auth 1000/min/user, recruiter Resdex 10/min/user
- Input validation via Zod at every API boundary
- Prisma-only DB access (no raw SQL except tagged template)
- File uploads: MIME-type allowlist, virus scan (ClamAV), signed-URL serving from R2
- Webhook endpoints: signature verification + IP allowlist
- Secrets in environment variables, never in repo

### 5.3 Reliability & Availability

- Target: 99.9% monthly uptime (43m downtime allowance).
- Error budget: 0.1% — automated alerts when consumed.
- Postgres: daily snapshot + 7-day PITR.
- Search index: daily reindex.
- File storage: R2 versioning + cross-region replication (production).

- All long-running jobs are idempotent and retryable.

## 5.4 Maintainability

- Single language (TypeScript) across stack — reduces context-switch cost.
- Monorepo with shared `packages/*` — types and queries reused across apps.
- Strict typecheck: `tsconfig.json` `strict: true`.
- Lint via ESLint + Prettier; pre-commit hooks.
- 80% unit-test coverage on `packages/*`; integration tests on critical flows.
- ADR (Architecture Decision Record) per major change in `docs/adr/`.

## 5.5 Portability

- All apps containerizable (Dockerfile per app).
- No vendor-locked APIs except: Stripe (replaceable with Razorpay), Cloudflare (replaceable with Akamai/Fastly), Vercel (replaceable with self-hosted Next.js + Node).

## 5.6 Scalability

| Stage        | Trigger     | What changes                                                                                                              |
|--------------|-------------|---------------------------------------------------------------------------------------------------------------------------|
| MVP          | 0-100k MAU  | Single Postgres, Meilisearch, Redis. Vercel hobby.                                                                        |
| Growth       | 100k-1M MAU | Postgres read replicas. Vercel Pro. Cloudflare Pro. PostHog growth tier.                                                  |
| Scale        | 1M-10M MAU  | Partition Postgres <code>Job</code> table by month. PgBouncer pooler. Meilisearch sharded or migration to Typesense.      |
| Naukri-scale | 10M+ MAU    | Switch search to Elasticsearch. Add Kafka for events. Multi-region active-active. CDN edge functions for personalization. |

## 5.7 Usability & Accessibility

- WCAG 2.1 Level AA (target 2.2 in Phase 2).
- Keyboard-navigable, screen-reader-friendly.
- All forms have visible labels and error messages with `aria-describedby`.
- Color contrast  $\geq 4.5:1$  body,  $\geq 3:1$  large text/graphics.
- `prefers-reduced-motion` respected.
- Focus rings always visible on keyboard interaction.

## 5.8 Internationalization

- Phase 1: English (Indian English idioms acceptable).

- Phase 3: Hindi + 5 regional languages (Tamil, Telugu, Kannada, Marathi, Bengali).
  - Currency: ₹INR only Phase 1; multi-currency with USD/AED for Gulf jobs Phase 3.
  - All strings extracted to `packages/ui/src/i18n/{lang}.json` from day 1, even if only English exists initially.
-

## 6. URL Architecture & Rendering Strategy

### 6.1 URL Slug Taxonomy

#### Naming rules

1. Lowercase only.
2. Hyphen-separated words.
3. **Numeric ID always at the end** (allows slug to drift without breaking links).
4. Multi-value path segments use `-and-` separator (sorted alphabetically).
5. Trailing slash: NEVER (configured via `trailingSlash: false`).

#### Pattern catalog

| Pattern                                    | Example                                                 | Folder                                                           | Render    |
|--------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------|-----------|
| <code>/</code>                             | <code>/</code>                                          | <code>app/page.tsx</code>                                        | PPR       |
| <code>/ {skill} -jobs</code>               | <code>/python-jobs</code>                               | <code>app/(seo-jobs)/[skill]-jobs/page.tsx</code>                | SSR       |
| <code>/jobs-in- {city}</code>              | <code>/jobs-in-bangalore</code>                         | <code>app/(seo-jobs)/jobs-in-[city]/page.tsx</code>              | SSR       |
| <code>/ {skill} -jobs-in- {city}</code>    | <code>/python-jobs-in-pune</code>                       | <code>app/(seo-jobs)/[skill]-jobs-in-[city]/page.tsx</code>      | SSR       |
| <code>/jobs-in- {c1} -and- {c2}</code>     | <code>/jobs-in-mumbai-and-pune</code>                   | same as <code>/jobs-in-[city]</code>                             | SSR       |
| <code>/job/ {slug} - {numId}</code>        | <code>/job/sales-executive-acme-12345</code>            | <code>app/job/[slug]/page.tsx</code>                             | SSR       |
| <code>/companies</code>                    | <code>/companies</code>                                 | <code>app/(seo-companies)/companies/page.tsx</code>              | SSR       |
| <code>/ {slug} -overview- {numId}</code>   | <code>/infosys-overview-13832</code>                    | <code>app/(seo-companies)/[slug]-overview-[id]/page.tsx</code>   | SSR       |
| <code>/working-at- {slug} - {numId}</code> | <code>/working-at-tcs-2114</code>                       | <code>app/(seo-companies)/working-at-[slug]-[id]/page.tsx</code> | SSR       |
| <code>/career-advice</code>                | <code>/career-advice</code>                             | <code>app/career-advice/page.tsx</code>                          | SSR       |
| <code>/career-advice/ {slug}</code>        | <code>/career-advice/how-to-write-a-cover-letter</code> | <code>app/career-advice/[slug]/page.tsx</code>                   | SSG + ISR |
| <code>/login , /register</code>            | —                                                       | <code>app/(auth)/*</code>                                        | CSR       |

| Pattern                                             | Example                  | Folder                        | Render              |
|-----------------------------------------------------|--------------------------|-------------------------------|---------------------|
| /profile,<br>/applications,<br>/saved-jobs, /alerts | —                        | app/(dashboard)/*             | CSR (auth-gated)    |
| /admin/*                                            | /admin/feature-flags     | app/admin/*                   | CSR (admin-gated)   |
| /pricing                                            | /pricing                 | app/pricing/page.tsx          | SSR (gated by flag) |
| /services/{slug}                                    | /services/resume-display | app/services/[slug]/page.tsx  | SSR (gated by flag) |
| /sitemap.xml,<br>/robots.txt                        | —                        | app/sitemap.ts, app/robots.ts | Generated           |

## 6.2 SSR/SSG/ISR/CSR Decision Matrix

The decision tree used for new pages:

- Does this page need to rank in Google?  
YES → SSR or SSG (must serve full HTML)  
NO → next step
- Same content for everyone?  
YES → SSG (build) IF rarely changes  
→ SSR + edge cache IF changes daily  
NO → CSR or RSC
- Too many possible URLs to pre-build (>10k)?  
YES → SSR + edge cache  
NO → SSG with on-demand revalidation
- Interactive form / dashboard?  
YES → CSR  
NO → SSR
- Auth-only?  
YES → noindex; CSR is fine

Rendering choice per page-type is in the §6.1 table.

## 6.3 Canonicalization Rules

- Slug drift:** when slug changes, OLD URL 301s to NEW.
- Trailing slash:** never; configured app-wide.
- Case:** lowercase only; middleware redirects uppercase.
- Query order:** alphabetical (improves cache hits).

5. **Self-referencing canonical**: every page sets `<link rel="canonical">` .
6. **Multi-value paths**: cities sorted alphabetically.

## 6.4 Sitemap Strategy

---

- Single `/sitemap.xml` until URL count exceeds 50,000.
  - At 50k+: use Next.js `generateSitemaps` to produce shards (`/sitemap-jobs/sitemap.xml` , `/sitemap-companies/sitemap.xml` , etc.) with index at `/sitemap.xml` .
  - Closed/expired jobs are excluded from sitemap.
  - Articles updated within last 30 days have `<changefreq>weekly</changefreq>` .
-

## 7. Subscription & Feature Flag System

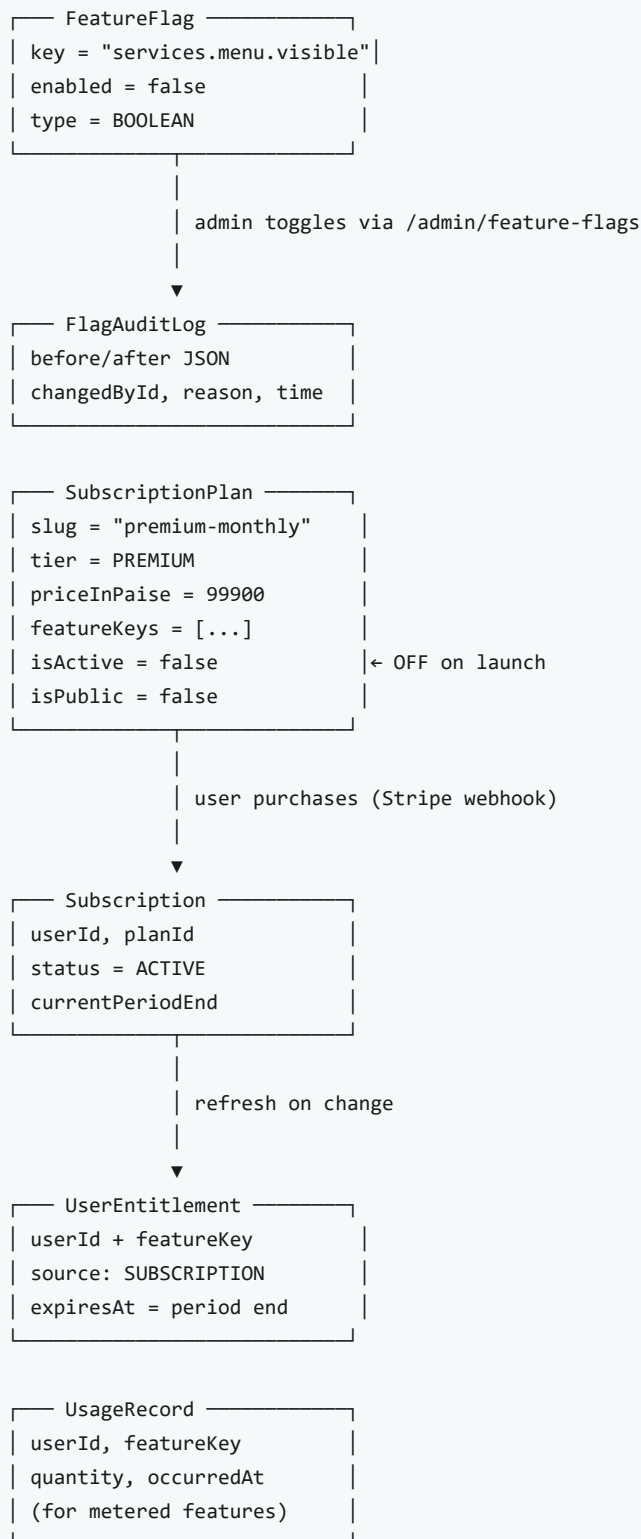
---

### 7.1 Goals

---

1. **Backend control of every paid feature.** A frontend deploy is NEVER required to launch or pause a paid feature. The admin flips a flag; the next page render reflects the new state.
2. **Freemium-by-default launch.** Every paid flag starts OFF. The Services menu is hidden on launch. `/pricing` returns 404. The product feels 100% free to all users.
3. **Per-user, per-cohort, per-tier control.** Flags can be toggled globally, gradually rolled out (10% → 50% → 100%), targeted at specific users, or gated by subscription tier.
4. **Auditable.** Every flag change is logged with before/after, user, timestamp, reason.
5. **Cache-friendly.** Flag evaluation is React-cached per request and Redis-cached cross-request.

## 7.2 Conceptual Model





## 7.3 Flag Types

| Type               | Use case                           | Evaluation                                                    |
|--------------------|------------------------------------|---------------------------------------------------------------|
| BOOLEAN            | Simple on/off                      | <code>flag.enabled</code>                                     |
| TIER_GATED         | Tied to subscription tier          | <code>user.tier in flag.requiredTiers</code>                  |
| PERCENTAGE_ROLLOUT | Gradual rollout (10% → 50% → 100%) | <code>hash(userId, flagKey) % 100 &lt; flag.percentage</code> |
| USER_TARGETED      | Specific user list (QA, beta)      | <code>userId in flag.targetUserIds</code>                     |
| COHORT_TARGETED    | A/B test cookie                    | <code>cohortCookie in flag.cohorts</code>                     |

## 7.4 Data Model

(Full Prisma schema in §8.)

```

FeatureFlag
├─ id, key (unique), description, uiLabel, category
├─ type, enabled
├─ percentage, targetUserIds, requiredTiers, cohorts
├─ createdAt, updatedAt, createdById, lastChangedById
├─ auditLogs[]

FlagAuditLog
├─ flagId, before (json), after (json), changedAt, changedById, reason

SubscriptionPlan
├─ id, slug (unique), name, description, tier
├─ priceInPaise, currency, intervalDays, trialDays
├─ featureKeys[]
├─ isActive, isPublic, sortOrder
├─ stripePriceId, razorpayPlanId
├─ subscriptions[]

Subscription
├─ id, userId, planId, status
├─ startedAt, currentPeriodStart, currentPeriodEnd
├─ cancelledAt, cancelAtPeriodEnd, cancelReason
├─ stripeSubscriptionId, razorpaySubscriptionId
├─ invoices[]

SubscriptionInvoice
├─ subscriptionId, amountInPaise, status, providerInvoiceId, paidAt

UserEntitlement
├─ userId + featureKey (unique), source, expiresAt, metadata

UsageRecord
├─ userId, featureKey, quantity, occurredAt, metadata

```

## 7.5 Evaluation Flow

```

async function evaluate(key: string, ctx: Context): Promise<{ enabled: boolean, reason: string }> {
  // 1. Admin grant (always wins)
  if (ctx.userId) {
    const grant = await prisma.userEntitlement.findUnique({
      where: { userId_featureKey: { userId: ctx.userId, featureKey: key } }
    });
    if (grant?.source === 'ADMIN_GRANT' && (!grant.expiresAt || grant.expiresAt > new Date())) {
      return { enabled: true, reason: 'admin_grant' };
    }
  }

  // 2. Look up flag
  const flag = await prisma.featureFlag.findUnique({ where: { key } });
  if (!flag || !flag.enabled) return { enabled: false, reason: 'flag_off' };

  // 3. Type-specific
  switch (flag.type) {
    case 'BOOLEAN':      return { enabled: true, reason: 'flag_on' };
    case 'USER_TARGETED': return { enabled: ctx.userId && flag.targetUserIds.includes(ctx.userId),
reason: 'user_targeted' };
    case 'TIER_GATED':    return { enabled: ctx.tier && flag.requiredTiers.includes(ctx.tier), reason:
'tier_match' };
    case 'PERCENTAGE_ROLLOUT': return { enabled: hash(ctx.userId, key) % 100 < flag.percentage, reason:
'percentage_in' };
    case 'COHORT_TARGETED': return { enabled: flag.cohorts.includes(ctx.cohort), reason: 'cohort_match' };
  }
}

```

## 7.6 Caching Strategy

- **In-process** (React `cache()` per request): same flag never hits the DB twice in one render tree.
- **Redis** (optional, cross-request): cache flag rows for 30s TTL. On flag change, publish invalidation via Redis pub/sub; subscribers (NestJS BFF, Next.js servers) clear local LRU.
- **Browser**: never expose flag values directly; only render output.

## 7.7 Admin UI

`/admin/feature-flags` page lists flags grouped by category:

- **services** — Services menu visibility, individual service flags
- **subscription** — Plans, pricing page visibility, billing system master
- **features** — Per-feature gating (TIER\_GATED)
- **recruiter** — Recruiter-specific flags
- **experiments** — A/B tests, percentage rollouts
- **killswitch** — Emergency kill switches

Each row: label · key · current state · toggle. Editing additional fields (percentage, target users, tiers) opens a side panel.

Audit log accessible at `/admin/audit-log?type=feature_flag`.

## 7.8 Initial Freemium State (Launch Day Configuration)

| Flag key                                               | Initial value            |
|--------------------------------------------------------|--------------------------|
| <code>services.menu.visible</code>                     | OFF                      |
| <code>services.resume_display.enabled</code>           | OFF                      |
| <code>services.resume_writing.enabled</code>           | OFF                      |
| <code>services.resume_writing_executive.enabled</code> | OFF                      |
| <code>services.ai_interview.enabled</code>             | OFF                      |
| <code>services.priority_applicant.enabled</code>       | OFF                      |
| <code>services.profile_spotlight.enabled</code>        | OFF                      |
| <code>services.recruiter_connect.enabled</code>        | OFF                      |
| <code>subscription.system.enabled</code>               | OFF                      |
| <code>subscription.pricing_page.visible</code>         | OFF                      |
| <code>subscription.plans.basic.enabled</code>          | OFF                      |
| <code>subscription.plans.premium.enabled</code>        | OFF                      |
| <code>subscription.plans.enterprise.enabled</code>     | OFF                      |
| <code>feature.unlimited_applications</code>            | OFF (TIER_GATED)         |
| <code>feature.profile_views_insights</code>            | OFF (TIER_GATED)         |
| <code>feature.salary_insights</code>                   | OFF (TIER_GATED)         |
| <code>feature.resume_download_pdf</code>               | OFF (TIER_GATED)         |
| <code>feature.bulk_apply</code>                        | OFF (TIER_GATED)         |
| <code>feature.ai_resume_review</code>                  | OFF (TIER_GATED)         |
| <code>recruiter.resdex.enabled</code>                  | OFF                      |
| <code>recruiter.bulk_messaging.enabled</code>          | OFF                      |
| <code>experiment.new_homepage</code>                   | OFF (COHORT_TARGETED)    |
| <code>experiment.ai_job_match</code>                   | OFF (PERCENTAGE_ROLLOUT) |
| <code>killswitch.job_alerts</code>                     | OFF                      |

| Flag key                     | Initial value |
|------------------------------|---------------|
| killswitch.resume_uploads    | OFF           |
| killswitch.new_registrations | OFF           |

| Plan slug         | isActive | isPublic |
|-------------------|----------|----------|
| basic-monthly     | false    | false    |
| premium-monthly   | false    | false    |
| premium-yearly    | false    | false    |
| enterprise-yearly | false    | false    |

## 7.9 Lifecycle: Enabling a Paid Service

Worked example — turning on Resume Display 3 months post-launch.

### Step 0: Prerequisites

The code path for Resume Display is already shipped (landing page, purchase API, Stripe price ID, success page). No deploy needed.

### Step 1: Internal QA test (USER\_TARGETED)

```
PATCH /api/admin/feature-flags/services.resume_display.enabled
{
  "type": "USER_TARGETED",
  "targetUserIds": [42, 43, 44],
  "enabled": true,
  "reason": "Internal QA"
}
```

Only users 42, 43, 44 see the service in the header dropdown.

### Step 2: Beta cohort (COHORT\_TARGETED)

```
PATCH /api/admin/feature-flags/services.resume_display.enabled
{
  "type": "COHORT_TARGETED",
  "cohorts": ["beta_2026_q3"],
  "reason": "Beta program"
}
```

Users with `cohort=beta_2026_q3` cookie see it.

### Step 3: 10% rollout

```
{
  "type": "PERCENTAGE_ROLLOUT",
  "percentage": 10,
  "reason": "Soft launch"
}
```

### Step 4: 50%, 100%

```
{ "percentage": 50 }    # then
{ "percentage": 100 }
```

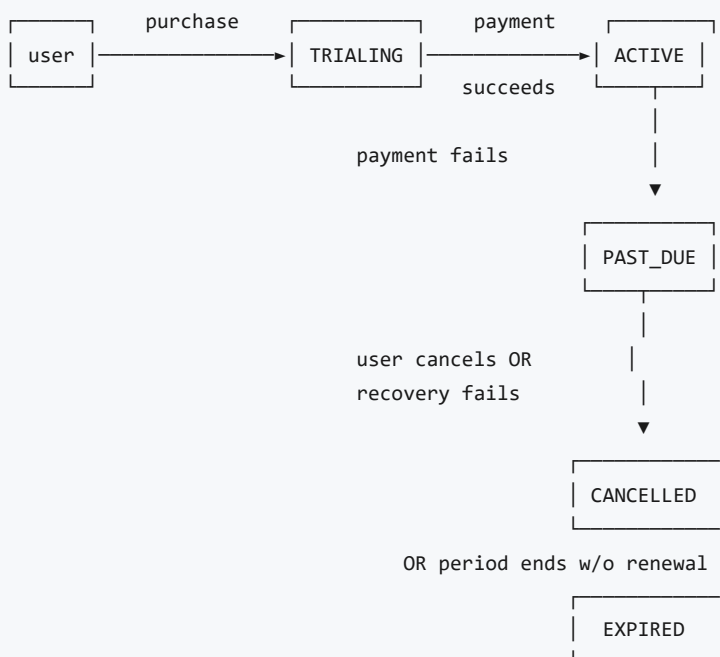
### Step 5: General availability

```
PATCH /api/admin/feature-flags/services.menu.visible
{ "enabled": true, "reason": "Services menu live" }

PATCH /api/admin/feature-flags/services.resume_display.enabled
{ "type": "BOOLEAN", "enabled": true, "reason": "GA launch" }
```

Now every visitor sees the Services menu with "Resume Display" item.

## 7.10 Subscription Lifecycle



State transitions are driven by Stripe webhook events. On every transition:

1. `Subscription.status` updated.

2. UserEntitlement rows recomputed.
3. Email notification sent.
4. Flag-eval cache invalidated for the user.

## 7.11 Payment Integration

- Provider: Stripe (primary), Razorpay (alternative for Indian-market preference).
- Integration mode: Stripe Checkout (hosted page) for v1; Stripe Elements (embedded) for v2.
- Webhooks: `customer.subscription.created`, `.updated`, `.deleted`, `invoice.payment_succeeded`, `invoice.payment_failed`.
- Webhook endpoint: `/api/webhooks/stripe`, signature-verified, idempotent.
- Refund handling via Stripe dashboard; mirror as `SubscriptionInvoice.status=REFUNDED`.

## 7.12 Entitlement Enforcement

Three layers — all required:

### Layer 1: Middleware (route gate)

```
// apps/web/middleware.ts
if (pathname.startsWith('/services/')) {
  const slug = pathname.split('/')[2];
  const flag = `services.${slug.replace(/-/g, '_')}.enabled`;
  if (!await isFlagEnabled(flag)) return redirect('/');
}
```

### Layer 2: Page (server component)

```
export default async function PricingPage() {
  if (!await isFlagEnabled(FLAG.PRICING_PAGE_VISIBLE)) notFound();
  ...
}
```

### Layer 3: API (server-only — non-bypassable)

```
export async function POST(req: NextRequest) {
  const user = await requireAuth();
  const allowed = await isFlagEnabled(FLAG.FEAT_BULK_APPLY, { userId: user.id });
  if (!allowed) return NextResponse.json({ error: 'Upgrade required' }, { status: 403 });
  ...
}
```

UI gating is for UX only. The API layer is the ONLY trusted enforcement point.

## 7.13 Audit & Observability

---

- Every `setFlag()` writes a `FlagAuditLog` row.
- Critical flag changes (`services.menu.visible`, `subscription.system.enabled`, `killswitches`) emit a Slack notification to `#feature-flag-changes`.
- Flag evaluations sampled at 1% rate to PostHog for "% of users seeing X" telemetry.

## 7.14 Worked Example: Launch Day → 6 Months Later

---

### Day 0 (launch):

- All flags OFF (except infrastructure-default).
- Header: Jobs · Companies · Career Advice · Login · Register.
- `/pricing` → 404.
- All `/services/*` → 404.
- All features behave as FREE.

### Day 30 (post-launch monitoring):

- Engineering team uses internal QA flags to test paid features end-to-end.
- No user-facing change.

### Day 90 (first service launch):

- Admin enables `services.menu.visible` AND `services.resume_writing.enabled` (BOOLEAN).
- Header now shows "Services → Resume Writing".
- `/services/resume-writing` page becomes accessible.
- One-time purchase flow live.

### Day 120 (subscription launch):

- Admin enables `subscription.system.enabled`, `subscription.pricing_page.visible`, `subscription.plans.premium.enabled`.
- Sets `SubscriptionPlan premium-monthly.isActive=true, isPublic=true`.
- Enables `feature.unlimited_applications`, `feature.salary_insights`, etc.
- `/pricing` shows Premium plan.
- Existing users see upgrade prompts on gated features.

### Day 180 (Resdex launch for recruiters):

- Admin enables `recruiter.resdex.enabled` (USER\_TARGETED for first cohort, then BOOLEAN).
- Recruiter portal shows Resume Database link.

Throughout: zero frontend deploys.

---

## 8. Data Model

### 8.1 Entity Relationship Overview



### 8.2 Schema Reference

The complete Prisma schema is at `packages/db/prisma/schema.prisma`. Highlights:



## Core entities

```

model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  passwordHash String
  name        String
  role        UserRole     @default(CANDIDATE)
  ...
  candidate   Candidate?
  recruiter   Recruiter?
  subscriptions Subscription[]
  entitlements UserEntitlement[]
  usageRecords UsageRecord[]
  ...
}

model Job {
  id          Int          @id @default(autoincrement())
  canonicalSlug String
  title       String
  description  String
  shortDescription String
  companyId   Int
  primaryCityId Int?
  cityIds     Int[]        // multi-location
  skillIds    Int[]
  status      JobStatus    @default(ACTIVE)
  postedAt    DateTime     @default(now())
  expiresAt   DateTime?
  ...
  @@index([status, postedAt])
  @@index([primaryCityId, status])
  @@index([companyId, status])
}

```

## Feature flag entities

```

model FeatureFlag {
  key          String    @unique
  description   String?
  type          FlagType @default(BOOLEAN)
  enabled       Boolean   @default(false)
  percentage    Int?
  targetUserIds Int[]
  requiredTiers SubscriptionTier[]
  cohorts       String[]
  category      String?
  uiLabel       String?
  ...
  auditLogs     FlagAuditLog[]
}

model SubscriptionPlan {
  slug          String      @unique
  name          String
  tier           SubscriptionTier
  priceInPaise  Int
  intervalDays  Int
  featureKeys   String[]
  isActive      Boolean     @default(false)
  isPublic      Boolean     @default(false)
  ...
}

model Subscription {
  userId        Int
  planId        Int
  status        SubscriptionStatus @default(TRIALING)
  currentPeriodStart DateTime
  currentPeriodEnd  DateTime
  stripeSubscriptionId String?    @unique
  ...
}

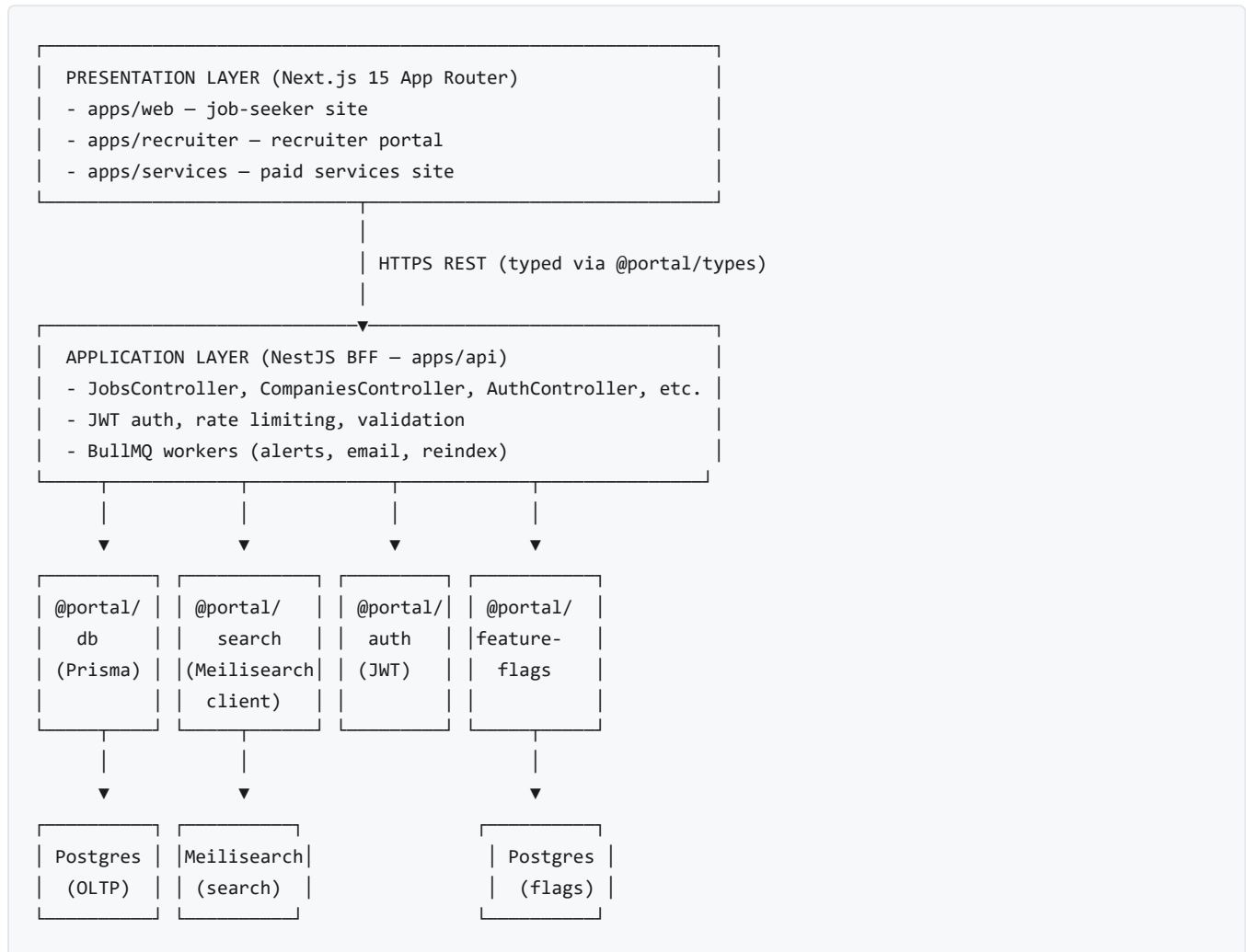
model UserEntitlement {
  userId      Int
  featureKey   String
  source      EntitlementSource
  expiresAt   DateTime?
  @@unique([userId, featureKey])
}

```

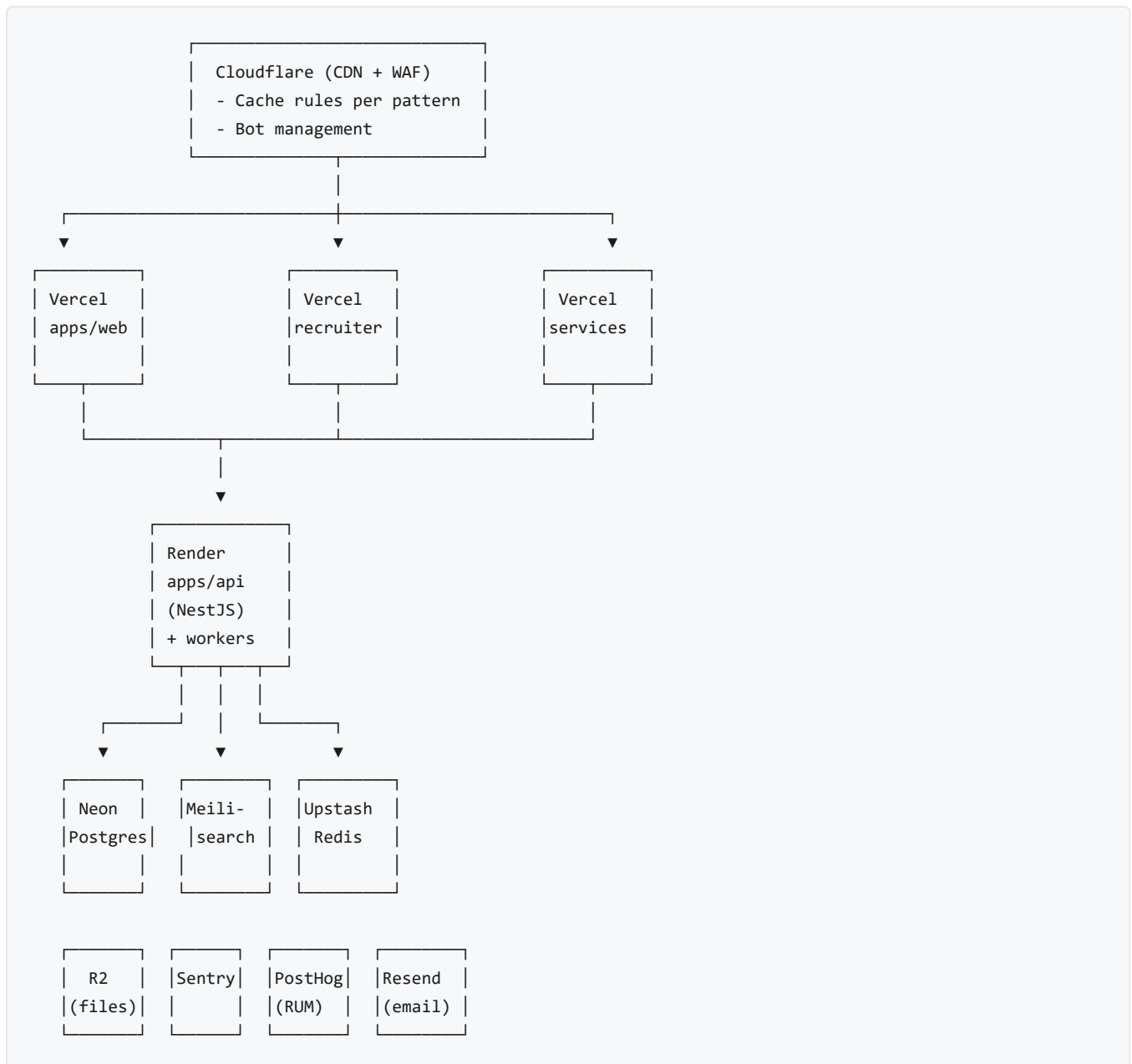
See [docs/database-schema.md](#) for design rationale (denormalization, indexing, partitioning roadmap).

## 9. System Architecture

### 9.1 Layered Architecture



## 9.2 Deployment Topology



## 9.3 Tech Stack

| Layer              | Tech                     | Why                                     |
|--------------------|--------------------------|-----------------------------------------|
| Frontend framework | Next.js 15 App Router    | SSR + RSC + PPR + ISR all in one        |
| UI library         | React 19                 | Server Components, concurrent rendering |
| Styling            | Tailwind CSS 3           | Atomic, fast, themeable                 |
| Forms              | Native + Zod             | No additional library needed            |
| State              | Zustand + TanStack Query | Lightweight, server-state separation    |
| Icons              | lucide-react             | Tree-shakeable, comprehensive           |

| Layer             | Tech                     | Why                                      |
|-------------------|--------------------------|------------------------------------------|
| Backend framework | NestJS                   | Modular, decorator-based, Spring-like DX |
| ORM               | Prisma                   | TS-first, migrations, studio             |
| Database          | PostgreSQL 16            | JSONB, GIN, partitioning, FTS            |
| Search            | Meilisearch              | Operationally simple, fast               |
| Cache / queues    | Redis (Upstash) + BullMQ | Standard combo                           |
| Object storage    | Cloudflare R2            | S3-compatible, no egress fees            |
| Email             | Resend                   | Modern, simple API                       |
| Auth              | Custom JWT + Argon2id    | Standard, no NextAuth dependency         |
| Payments          | Stripe                   | When activated                           |
| CDN + DNS + WAF   | Cloudflare               | Free tier viable, Workers for edge logic |
| Frontend hosting  | Vercel                   | Native Next.js support                   |
| Backend hosting   | Render or Fly.io         | Long-running container, queues           |
| Monitoring        | Sentry + PostHog         | Errors + product analytics               |
| CI/CD             | GitHub Actions           | Standard                                 |

## 9.4 Data Flow — Sample Request

A user visits `https://your-domain.com/python-jobs-in-pune`:

1. Cloudflare edge (Mumbai PoP):

- Cache lookup. If HIT (within 5-min TTL): return cached HTML in ~10ms.
- If MISS: forward to Vercel.

2. Vercel (ap-south-1 / Singapore):

- Next.js routes to `app/(seo-jobs)/[skill]-jobs-in-[city]/page.tsx`.
- `params = { skill: 'python', city: 'pune' }`.
- `generateMetadata()` fetches skill+city for title/description.
- Page Server Component:
  - `getSkillBySlug('python')` → Postgres
  - `getCityBySlug('pune')` → Postgres
  - `searchJobs({ skill, cities: [pune.id] })` → Meilisearch
- All three run in parallel via `Promise.all`.
- Renders RSC stream → ~150KB HTML.

3. Cloudflare caches the response per cache rule.

4. Browser:

- Receives streamed HTML, paints.
- Hydrates React Client Components (filter dropdowns, save buttons).
- Fires beacons to PostHog and Sentry.

Total latency target: 800ms p75 first paint, 2.5s p75 LCP.

---

# 10. Project Structure

## 10.1 Monorepo Layout

```

my-job-portal/
├─ apps/
│   └─ web/                                # Next.js 15 - main job-seeker site
│       └─ app/
│           ├── (marketing)/
│           ├── (seo-jobs)/
│           │   ├── [skill]-jobs/
│           │   ├── jobs-in-[city]/
│           │   └─ [skill]-jobs-in-[city]/
│           ├── (seo-companies)/
│           │   ├── companies/
│           │   ├── [slug]-overview-[id]/
│           │   └─ working-at-[slug]-[id]/
│           ├── job/[slug]/
│           ├── career-advice/[slug]/
│           ├── (auth)/login,register/
│           ├── (dashboard)/profile,applications,saved-jobs,alerts/
│           ├── admin/
│           │   └─ feature-flags/
│           ├── pricing/
│           ├── services/[slug]/
│           ├── api/
│           │   ├── jobs/
│           │   ├── auth/login/
│           │   └─ admin/feature-flags/[key]/
│           ├── sitemap.ts
│           ├── robots.ts
│           └─ layout.tsx
│       ├── components/
│       │   ├── job/, company/, layout/, dashboard/, admin/, article/
│       ├── lib/
│       │   ├── flags/index.ts
│       │   └─ seo/json-ld.ts
│       ├── middleware.ts
│       ├── next.config.mjs
│       ├── tailwind.config.ts
│       └─ package.json
│   └─ recruiter/                          # Recruiter portal (Next.js)
│   └─ services/                          # Paid services site (Next.js)
│   └─ api/                               # NestJS BFF
│       ├── src/
│       │   ├── main.ts, app.module.ts
│       │   ├── jobs/, companies/, auth/, candidates/, recruiters/
│       │   ├── feature-flags/, subscriptions/
│       │   └─ alerts/alerts.processor.ts
│       └─ package.json
├─ packages/
│   └─ ui/                                # Design system
│       └─ src/atoms/, molecules/, utils/

```

```

├── db/                                # Prisma schema + queries
│   ├── prisma/schema.prisma
│   ├── prisma/seed.ts
│   ├── prisma/seed/flags.ts
│   └── src/queries/
├── search/                            # Meilisearch wrapper
│   └── scripts/reindex.ts
├── auth/                              # JWT + refresh tokens
├── feature-flags/                     # ★ Backend-controlled flags
│   └── src/index.ts
└── types/                             # Shared TS types + Zod schemas

├── docs/
│   ├── architecture.md
│   ├── ssr-csr-decisions.md
│   ├── url-taxonomy.md
│   ├── subscription-system.md       # ★ Subscription deep-dive
│   ├── database-schema.md
│   └── deployment.md
├── package.json                      # Workspace root (pnpm workspaces)
├── turbo.json                       # Build orchestration
├── tsconfig.json                    # Path aliases for @portal/*
├── .env.example
└── README.md

```

## 10.2 App Boundaries

| App            | Subdomain               | Purpose               | Render                 |
|----------------|-------------------------|-----------------------|------------------------|
| apps/web       | www.your-domain.com     | Job-seeker portal     | Mostly SSR + some CSR  |
| apps/recruiter | recruit.your-domain.com | Recruiter tools       | Mostly CSR (post-auth) |
| apps/services  | resume.your-domain.com  | Paid services landing | SSR                    |
| apps/api       | api.your-domain.com     | BFF for all clients   | API                    |



## 10.3 Package Dependencies

```
apps/web —┬─ @portal/ui
          │├─ @portal/db
          │├─ @portal/search
          │├─ @portal/auth
          │├─ @portal/types
          └─ @portal/feature-flags

apps/recruiter —┬─ @portal/ui
                 │├─ @portal/db
                 │├─ @portal/search
                 │├─ @portal/auth
                 └─ @portal/types

apps/services —┬─ @portal/ui
                 │├─ @portal/auth
                 └─ @portal/types

apps/api —┬─ @portal/db
           │├─ @portal/search
           │├─ @portal/auth
           │├─ @portal/types
           └─ @portal/feature-flags

packages/feature-flags —┬─ @portal/db
                        └─ @portal/types

packages/auth —┬─ @portal/db
                └─ @portal/types

packages/search —┬─ @portal/db
                  └─ @portal/types
```

# 11. Deployment & Operations

## 11.1 Hosting

| Service            | Provider                       | Cost (initial)    |
|--------------------|--------------------------------|-------------------|
| Frontends (3 apps) | Vercel                         | Hobby (free)      |
| API + workers      | Render                         | \$7-25/mo         |
| Postgres           | Neon                           | Free 0.5GB        |
| Search             | Meilisearch self-hosted on Fly | \$5/mo            |
| Cache + queues     | Upstash Redis                  | Free 10k cmd/day  |
| Object storage     | Cloudflare R2                  | Free 10GB         |
| CDN + WAF          | Cloudflare                     | Free              |
| Errors             | Sentry                         | Free 5k events/mo |
| Analytics          | PostHog Cloud                  | Free 1M events/mo |
| Email              | Resend                         | Free 3k/mo        |

Estimated total at 10k MAU: **\$0-20/month**.

## 11.2 CI/CD Pipeline

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - actions/checkout@v4
      - pnpm/action-setup@v3 (version: 9)
      - actions/setup-node@v4 (node-version: 20, cache: pnpm)
      - pnpm install --frozen-lockfile
      - pnpm lint
      - pnpm test
      - pnpm build
      - (only on main) Vercel auto-deploy
      - (only on main) Render redeploy via webhook
```

Database migrations run separately in CI:

```
DATABASE_URL=$PROD_DB pnpm prisma migrate deploy
```

Use **expand-then-contract** pattern for breaking changes:

1. Deploy: add new column (nullable)
2. Backfill via data migration
3. Deploy: app uses new column
4. Deploy: drop old column

## 11.3 Monitoring & Observability

| Concern      | Tool                                 | Detail                                                |
|--------------|--------------------------------------|-------------------------------------------------------|
| Errors       | Sentry                               | FE + API; tag every release with git SHA              |
| Tracing      | Sentry Performance                   | Distributed traces via <code>x-trace-id</code> header |
| Logs         | Cloudflare Logpush → R2              | Structured JSON archival                              |
| Metrics      | Prometheus exporter on API → Grafana | API latency, DB pool, queue depth                     |
| Web Vitals   | PostHog or Vercel Analytics          | LCP, INP, CLS p75 dashboards                          |
| Uptime       | BetterStack / Checkly                | <code>/api/health</code> 5xx alerts                   |
| Flag changes | Slack webhook                        | Posts to <code>#feature-flag-changes</code>           |

Alerts:

- API p95 > 500ms for 5 minutes → page on-call.
- Error rate > 1% for 5 minutes → page on-call.
- DB CPU > 80% sustained 10 minutes → warning.
- Postgres replication lag > 30s → warning.
- Stripe webhook failure rate > 5% → page on-call.

## 11.4 Disaster Recovery

| Asset        | Strategy                                 | RTO | RPO                    |
|--------------|------------------------------------------|-----|------------------------|
| Postgres     | Daily snapshot + 7-day PITR              | 1h  | 5m                     |
| Search index | Daily reindex from Postgres              | 30m | up to 24h (acceptable) |
| Resume files | R2 versioning + cross-region replication | 1h  | 0                      |
| Code         | GitHub                                   | n/a | n/a                    |
| Secrets      | Vercel/Render env vars + 1Password vault | 1h  | n/a                    |

Runbooks live at `docs/runbooks/{incident-type}.md` (TBC).

## 12. Roadmap

---

### 12.1 Phase 1 — Freemium MVP (Months 0-3)

---

**Goal:** validated freemium product with strong SEO, growing user base.

- ☒ Job search & SRPs (skill, city, combo, multi-city)
- ☒ Job detail with JSON-LD
- ☒ User registration, login, profile, resume upload
- ☒ Apply, save jobs, alerts
- ☒ Company directory + profiles
- ☒ Career advice CMS
- ☒ Recruiter portal: post jobs, view applicants
- ☒ Sitemap + robots
- ☒ Admin: feature flags, user lookup
- ☒ Subscription system **built but every flag OFF** (no monetization yet)

**Success metrics:**

- 10k MAU by month 3
- Top 3 rank for "{city} jobs" in at least 5 cities
- 60% LCP under 2.5s

### 12.2 Phase 2 — Subscription Launch (Months 3-6)

---

**Goal:** monetize a small fraction of engaged users.

- Enable `services.menu.visible` + 2-3 services (Resume Display, Resume Writing).
- Enable `subscription.system.enabled`, launch Premium plan.
- Stripe integration (or Razorpay).
- AI Interview product (third-party API integration).
- Job alert email refinement (A/B test subject lines).
- Multi-language: Hindi.

**Success metrics:**

- 1-3% paid conversion of MAU.
- ₹10L MRR by month 6.

### 12.3 Phase 3 — Enterprise & Scale (Months 6-12)

---

**Goal:** B2B products and infrastructure for scale.

- Recruiter Resdex (resume database).
- Enterprise plan (₹50K+/yr).

- Resume parsing AI.
  - Internationalization: 5 regional languages.
  - ATS API for partners.
  - Native mobile apps (Phase 3.5).
-

# Appendix A — Glossary

---

| Term              | Definition                                                                            |
|-------------------|---------------------------------------------------------------------------------------|
| BFF               | Backend-for-Frontend; one API service shared by all client surfaces.                  |
| Canonicalization  | Ensuring that one piece of content has exactly one canonical URL.                     |
| Cohort            | A group of users (often by signup date or A/B assignment) tracked together.           |
| Edge cache        | A CDN-side cache layer (Cloudflare, Akamai) that serves cached HTML close to users.   |
| Entitlement       | A feature a user has access to, computed from their subscription + admin grants.      |
| Feature flag      | A backend-controlled toggle that determines whether a code path or feature is active. |
| Freemium          | A pricing model where a base product is free; paid tiers add features.                |
| GIN index         | Postgres index type for arrays and full-text search.                                  |
| ISR               | Incremental Static Regeneration; SSG with periodic background revalidation.           |
| Killswitch        | A flag designed for emergency disabling (e.g., killswitch.job_alerts).                |
| Materialized view | A pre-computed query result stored as a table for fast reads.                         |
| Metered feature   | A feature with usage limits (e.g., 10 applications/day).                              |
| Middleware        | Code that runs on every request before the route handler (Next.js middleware.ts).     |
| PPR               | Partial Pre-rendering; mixing static shell with dynamic streamed islands.             |
| Prefetch          | Browser fetches a resource speculatively before user navigates.                       |
| RSC               | React Server Component; renders on server, no JS shipped to browser.                  |
| Slug              | Human-readable URL component (e.g., "python-jobs").                                   |
| SRP               | Search Results Page.                                                                  |
| SSR               | Server-Side Rendering; HTML built at request time.                                    |
| SSG               | Static Site Generation; HTML built at build time.                                     |
| Tier              | Subscription level (FREE, BASIC, PREMIUM, ENTERPRISE).                                |
| TTL               | Time-to-live; how long a cached value is considered fresh.                            |

## Appendix B — Naukri.com Reference Findings (Verified May 2026)

---

This SRS draws on a reverse-engineering report of Naukri.com performed by direct HTTP probing and analysis of public Naukri Engineering articles. Key facts:

1. **Hybrid stack**, NOT a single Next.js app. Five distinct front-end shells coexist on the apex domain (`/s/9/121/` Next.js 13.4, `/s/7/103/` legacy React, `/s/7/109/` companies, `/s/7/123/` career-advice, `/s/4/101/` ancient jQuery).
2. **SEO landings** (`/skill-jobs`, `/jobs-in-city`) and **JD pages** are served by Next.js 13.4 App Router with RSC streaming.
3. **Companies hub** (`/slug-overview-id`, `/working-at-slug-id`) runs on legacy PHP/Symfony.
4. `/companies-hub` **actually returns 404**; the canonical companies directory is `/allcompanies` (Express SSR).
5. `/blog` runs on Ghost 5.35; `/campus` runs on WordPress; `/code360` is hosted on Amazon S3 + CloudFront.
6. `insights.naukri.com` is a Tableau Server deployment, not custom code.
7. **Job detail URL pattern**: `/job-listings-title-company-city-exp-numericId` — numeric ID at the end is the DB primary key.
8. **Robots.txt blocks AI crawlers** (claudebot, gptbot, perplexitybot, etc.) from JD/SRP pages but allows them on `/blog/`, `/career-advice/`, `/code360/`.
9. **Akamai CDN** sits in front; origin returns `Cache-Control: private, no-cache` — Akamai overrides via cache rules.
10. **Backend**: Java/Spring Boot microservices, MySQL (master-slave + sharding), Elasticsearch (100 indexes × 1 shard each via application-layer routing), Kafka + Logstash + Kibana.
11. **Recruiter side** uses custom auth headers (`API-ID`, `API-TOKEN`, `clientId`, etc.) — not standard JWT.
12. **Multi-city URLs are unbounded**: `/jobs-in-mumbai-and-pune-and-bangalore-and-delhi-ncr` returns 200.

We adopt: hybrid SSR/CSR split, slug+ID URL pattern, sitemap-driven SEO, multi-city URL combinator, JSON-LD on JDs, edge-cache-driven freshness.

We modernize: single Next.js stack instead of 5 shells, NestJS BFF instead of Java, PostgreSQL instead of MySQL, Meilisearch instead of Elasticsearch, Cloudflare instead of Akamai, standard JWT instead of custom headers.

Full report: `Research/naukri-deep-dive-analysis.md`.

---

## Appendix C — Risk Register

| #   | Risk                                                           | Severity | Likelihood | Mitigation                                                                 |
|-----|----------------------------------------------------------------|----------|------------|----------------------------------------------------------------------------|
| R1  | Postgres single-point-of-failure                               | HIGH     | LOW        | Daily snapshot + PITR; managed Neon (auto-failover)                        |
| R2  | Stripe webhook lost → entitlement not granted                  | MEDIUM   | MEDIUM     | Webhook signature verification + idempotency + reconciliation cron         |
| R3  | Flag change accidentally exposes paid feature                  | MEDIUM   | LOW        | Audit log + Slack notification on every change; confirm dialog in admin UI |
| R4  | SEO landing page generates wrong canonical → duplicate content | HIGH     | MEDIUM     | Strict canonicalization rules; weekly Search Console review                |
| R5  | JD JSON-LD invalid → de-listed from Google for Jobs            | HIGH     | LOW        | CI check via Google Rich Results validator                                 |
| R6  | Cloudflare cache poisoning                                     | HIGH     | LOW        | Strict CSP + custom cache key including auth state                         |
| R7  | Resume upload contains malware                                 | HIGH     | LOW        | ClamAV scan before serving; signed-URL-only access                         |
| R8  | Recruiter scrapes candidate database                           | HIGH     | MEDIUM     | Resdex rate limiting; mask identity until "purchase contact"; audit log    |
| R9  | Search index out of sync with DB                               | MEDIUM   | MEDIUM     | Daily full reindex + LISTEN/NOTIFY incremental sync                        |
| R10 | Performance regression on key page                             | MEDIUM   | MEDIUM     | Lighthouse CI in PRs; performance budgets enforced                         |



## Appendix D — Compliance Checklist

---

| Item                                                     | Status                                 |
|----------------------------------------------------------|----------------------------------------|
| HTTPS-only with HSTS preload                             | ✓ Required                             |
| Cookie consent banner (where required by jurisdiction)   | TBD per legal                          |
| Privacy policy page ( /privacypolicy )                   | ✓ Required                             |
| Terms & conditions page ( /termsconditions )             | ✓ Required                             |
| GDPR-style data export (user data download)              | ✓ Phase 2                              |
| GDPR-style data deletion ("right to be forgotten")       | ✓ Phase 2                              |
| Audit log for admin actions                              | ✓ Built-in (FlagAuditLog + extendable) |
| Email unsubscribe link in every marketing email          | ✓ Required                             |
| WCAG 2.1 Level AA conformance                            | ✓ Target                               |
| PCI-DSS scope minimization (no card data on our servers) | ✓ Stripe-hosted checkout               |

# Appendix E — Sample Feature Flag Configurations

---

## E.1 Launch day — minimal flag set (auto-seeded by `pnpm db:seed`)

All 26 flags from §7.8 created with `enabled: false`. All 4 plans created with `isActive: false`.

## E.2 Internal QA for Resume Display

```
{
  "key": "services.resume_display.enabled",
  "type": "USER_TARGETED",
  "targetUserIds": [42, 43, 44],
  "enabled": true,
  "reason": "Internal QA round 1"
}
```

## E.3 10% percentage rollout of new homepage variant

```
{
  "key": "experiment.new_homepage",
  "type": "PERCENTAGE_ROLLOUT",
  "percentage": 10,
  "enabled": true,
  "reason": "A/B test against baseline"
}
```

## E.4 Premium-tier-only feature

```
{
  "key": "feature.salary_insights",
  "type": "TIER_GATED",
  "requiredTiers": ["PREMIUM", "ENTERPRISE"],
  "enabled": true,
  "reason": "GA — Premium tier perk"
}
```

## E.5 Emergency killswitch

```
{
  "key": "killswitch.job_alerts",
  "type": "BOOLEAN",
  "enabled": true,
  "reason": "Resend.com outage — pause all alert emails until resolved"
}
```

When this is ON, the BullMQ alerts processor skips sending emails (checks the flag at start of every job).

## E.6 Subscription launch sequence (when ready)

```
# Order matters: turn on infra flags first, then make plans visible.

# 1. Master switch
PATCH /api/admin/feature-flags/subscription.system.enabled
{ "enabled": true, "reason": "Subscription launch" }

# 2. Make pricing page accessible
PATCH /api/admin/feature-flags/subscription.pricing_page.visible
{ "enabled": true }

# 3. Enable specific plan
PATCH /api/admin/feature-flags/subscription.plans.premium.enabled
{ "enabled": true }

# 4. Mark plan as sellable in DB
UPDATE "SubscriptionPlan"
SET "isActive" = true, "isPublic" = true
WHERE slug = 'premium-monthly';

# 5. Enable features included in the plan (TIER_GATED)
PATCH /api/admin/feature-flags/feature.unlimited_applications
{ "enabled": true }
PATCH /api/admin/feature-flags/feature.salary_insights
{ "enabled": true }
# ... etc

# 6. Verify
- /pricing now lists Premium
- Existing free users see upgrade prompts
- Stripe checkout flow works end-to-end
```

### End of document

For ongoing changes, edit this SRS in `Research/SRS.md` and regenerate the PDF via `node Research/build-srs-pdf.mjs`.